Argus-SQL: Finding the Accuracy–Latency Sweet Spot in Natural Language and Voice-to-SQL via Value-Grounded Foreign-Key Closure and Semantic Invariant Verification

Independent Research Manuscript

Clean-Room Open Research Framework (`argus-sql v0.1.0`)

Personal scholarly work; unaffiliated with and unendorsed by any employer.

November 2026

Abstract

Natural Language to SQL (NL2SQL) stands poised to become the foundational execution substrate of conversational and voice-driven user interfaces over enterprise-scale relational data. Yet a rigorous gap separates current academic leaderboard pipelines on benchmarks such as BIRD-SQL from interactive, voice-ready deployment: monolithic single-pass prompts collapse under schema overload and dirty cell literals, whereas brute-force multi-agent ensembles achieve high Execution Accuracy (EX) only by sampling dozens of homogeneous completions at prohibitive token cost and multi-second latency—while remaining vulnerable to correlated dialect hallucinations.

In this paper, we conduct a systematic compositional study of modern NL2SQL architectures (`DIN-SQL`, `DAIL-SQL`, `MAC-SQL`, `MCS-SQL`, and `CHESS`) and isolate five structural failure modes that govern real-world relational querying: (F1) spoken ASR disfluencies and colloquial quantifiers, (F2) dirty database cell-value drift, (F3) foreign-key bridge-table pruning collapse, (F4) silent SQLite dialect bugs (integer-division truncation and unfiltered `NULL` extrema), and (F5) naive majority-voting tyranny. By recombining the strongest orthogonal operators from prior systems and introducing deterministic post-execution verification, we propose **Argus-SQL**, a four-stage hybrid architecture uniting: (1) a *Voice & Intent Canonicalizer*, (2) an offline *Character 3-Gram Value Index paired with Foreign-Key Discounted Steiner Closure*, (3) *Complexity-Gated Diverse Batch Synthesis* ($K = 1$ fast-path routing for simple utterances vs. $K = 3$ orthogonal strategies for analytical queries), and (4) a *Semantic Invariant Verifier with Zero-Token Algebraic Dialect Repair and Invariant-Weighted Execution Consensus*. On our controlled 48-query `Mini-BIRD` and spoken `Voice-BIRD` stress-test suite, **Argus-SQL** improves Execution Accuracy monotonically over zero-shot, decomposed, execution-refiner, and value-grounded self-consistency baselines (from 37.5% to 100.0% on Clean and from 16.7% to 100.0% on Voice; paired exact McNemar $p = 5.96 \times 10^{-8}$) while reducing simple-query token overhead via adaptive routing. We release our clean-room, zero-dependency modular codebase and turnkey `BIRD-Dev` evaluation harness to accelerate reproducible NL2SQL systems research.

1 Introduction

The emergence of capable Large Language Models (LLMs) has catalyzed a fundamental shift in human–computer interaction: rather than navigating rigid dashboards or hand-authoring analytical queries, users increasingly expect to express intent directly in natural language—and, as voice becomes a primary ambient interface, through spontaneous spoken utterances. Within this paradigm, Natural Language to SQL (NL2SQL) occupies a uniquely high-leverage position. Unlike free-form retrieval-augmented generation, which struggles to aggregate millions of records faithfully, translating user intent into executable relational algebra allows a conversational interface to query gigabytes or terabytes of structured data with mathematical precision.

Early cross-domain benchmarks such as Spider [12] demonstrated that neural models could master textbook relational syntax over clean, compact schemas. However, the introduction of the **BIRD-SQL** benchmark [6] exposed how severely textbook assumptions break down on realistic databases characterized by wide tables, noisy and abbreviated string cells, cryptic column identifiers, multi-hop foreign-key graphs, and domain-specific arithmetic formulas (*external evidence*). When spoken voice interfaces are layered atop such databases, Automatic Speech Recognition (ASR) disfluencies (*"uh," "you know"*), word stutters, spoken cardinals (*"top five"*), and colloquial shorthand (*"Calif," "LA county"*) compound schema and value ambiguity even further.

Recent leaderboard entries have pushed BIRD Execution Accuracy steadily upward through decomposed Chain-of-Thought prompting [11, 7], code-style DDL representations [3], multi-agent syntax refiners [9], locality-sensitive cell retrieval [8], and large-sample self-

consistency ensembles [4, 5]. Yet practitioners and reviewers confront two unresolved scientific questions:

1. **Component Attribution**: Which specific architectural primitives actually drive accuracy gains across each failure mode, and where do popular subsystems silently fail or cancel one another out?
2. **The Voice-Ready Sweet Spot**: How can we synthesize a hybrid architecture that approaches multi-sample leaderboard accuracy *without* incurring 20× LLM sampling costs or succumbing to correlated dialect errors?

Contributions. This work tells an empirical, engineering-grounded story from baseline failure diagnosis to hybrid system synthesis:

- **Five-Mode Failure Taxonomy (§2)**: We formalize five recurring failure modes spanning spoken intent drift, dirty cell literals, broken foreign-key join subgraphs, silent SQLite arithmetic/`NULL` semantics, and majority-voting tyranny.
- **FK-Discounted Steiner Schema Linking (§3.2)**: We show that naive lexical linking suffers a dual pathology—exploding on star schemas via shared `*_id` keys while dropping lexically opaque bridge tables—and resolve both via surrogate-key seed discounting plus shortest-path foreign-key closure paired with character 3-gram value indexing.
- **Semantic Invariant Verification & Zero-Token Algebraic Repair (§3.4)**: Moving beyond syntax-only refiners [9, 1], **Argus-SQL** audits execution results against seven dialect and intent invariants, deterministically repairing SQLite integer truncation across `SELECT` and `ORDER BY` clauses, guarding ascending `NULL` extrema, and deduplicating 1-to-many join fanouts in < 1 ms with zero extra LLM tokens.
- **Composable Clean-Room Benchmark Suite (§4)**: We implement five progressive architectures (`B0`–`B3` and **Argus-SQL**) with leave-one-out ablation flags, evaluate them across clean and spoken `Voice-BIRD` perturbations, and provide an end-to-end reproducible runner for official `BIRD-Dev`.

2 Anatomy & Failure Taxonomy of NL2SQL

Table 1 synthesizes the design space of modern public BIRD-SQL systems. Rather than viewing these systems as isolated black boxes, we decompose an NL2SQL pipeline into four sequential operators over a database schema $\mathcal{S} = (\mathcal{T}, \mathcal{E}_{\text{FK}})$, database instance $\mathcal{D}$, external evidence e, and user utterance u:

$$\hat{q} = \mathcal{V}_{\text{arb}}\Big(\mathcal{G}_{\text{syn}}\big(\mathcal{L}_{\text{link}}(\mathcal{S}, \mathcal{D}, \mathcal{C}_{\text{int}}(u, e))\big), \mathcal{D}\Big), \qquad (1)$$

where $\mathcal{C}_{\text{int}}$ canonicalizes user intent, $\mathcal{L}_{\text{link}}$ grounds cell values and prunes the schema graph, $\mathcal{G}_{\text{syn}}$ synthesizes K SQL candidates, and $\mathcal{V}_{\text{arb}}$ executes, repairs, and arbitrates among candidates. Inspecting errors across existing instantiations of Eq. (1) reveals five orthogonal failure modes:

F1: Spoken Disfluency & Quantifier Drift. In voice-first interfaces, ASR transcripts contain conversational preambles (*"uh hey can you please tell me. . . "*), mid-clause fillers (*"you know"*), word stutters, and spoken number tokens (*"top three," "first five"*). Uncleaned transcripts pollute entity-span extraction and cause limit/complexity parsers to misclassify analytical queries.

F2: Dirty Database Cell-Value Drift. Users routinely utter abbreviations or colloquial variants (*"Calif," "LA county," "NY"*), whereas physical tables store normalized strings (`'California'`, `'Los Angeles'`, `'New York'`). Systems lacking a database cell index [7, 3, 9] emit syntactically valid predicates such as `WHERE state = 'Calif'` that silently evaluate to zero rows.

F3: Star-Schema Overload vs. Bridge-Table Collapse. Wide schemas containing legacy or staging tables (*e.g.*, `frpm_legacy` alongside `frpm`) confuse zero-shot full-DDL prompts (`B0`). Conversely, pure lexical column pruning (`B1`) fails in two opposite ways: (i) if foreign-key columns such as `client_id` receive full lexical credit when the user mentions *"clients,"* every child table in a star schema is falsely retained; (ii) if intermediate junction tables carry lexically opaque names (*e.g.*, `account_disp` connecting `clients` to `loans`), lexical pruning drops the bridge table entirely, rendering a valid `JOIN` impossible.

F4: Silent SQLite Dialect & Formula Bugs. Execution-guided refiners such as `MAC-SQL` [9] trigger *only* when SQLite raises a runtime exception or returns an empty result set. On BIRD-SQL, however, two pervasive SQLite semantics produce non-empty, exception-free—yet completely wrong—answers:

1. *Integer-Division Truncation*: In SQLite, dividing two `INTEGER` expressions (`SUM(free_meals) / SUM(enrollment)`) truncates toward zero, returning `0` instead of `0.3125`—and if `ORDER BY a / b DESC LIMIT k` omits `REAL` casting, all ratios below 1.0 tie at `0`, corrupting top-k rankings.
2. *Unfiltered `NULL` Ascending Extrema*: SQLite treats `NULL` as smaller than any numeric value. Consequently, `ORDER BY sat_score ASC LIMIT 1` silently returns a row with `NULL` rather than the minimum recorded score whenever `WHERE sat_score IS NOT NULL` is omitted.

Table 1: Architectural decomposition and published public BIRD-SQL Dev Execution Accuracy (EX %) of representative peer-reviewed and arXiv NL2SQL frameworks [6, 7, 3, 9, 8, 4]. **Argus-SQL** selectively recombines compact DDL, 3-gram cell grounding, and FK-closure linking while replacing high-K naive voting and syntax-only refinement with adaptive routing ($K \in \{1, 3\}$), semantic invariant repair, and weighted consensus.

Published System (Public Literature)	Voice Canon.	Cell Value Index	FK-Closure Pruning	Candidate Budget K	Refinement / Selection	Backbone (Reported)	Published BIRD Dev EX (%)
C3-SQL [2]	×	×	Lexical	$K = 10$–20	Naive Exec. Vote	GPT-3.5	50.2
DIN-SQL [7]	×	×	Lexical	$K = 1$	Self-Correction	GPT-4	50.7
DAIL-SQL [3]	×	×	Full DDL	$K = 1$–5	Self-Consistency	GPT-4	54.8–55.9
MAC-SQL [9]	×	×	Selector	$K = 1$	Syntax/Empty Refiner	GPT-4	59.4
MCS-SQL [4]	×	Prompted	Multi-Prompt	$K = 20$	Multiple-Choice + Vote	GPT-4	63.4
CHESS [8]	×	LSH / MinHash	Col. Pruner	Multi-Unit	Unit-Test Checker	GPT-4 / Open	65.0–68.3
Argus-SQL (Ours)	✓	**3-Gram Inv.**	**Steiner FK**	**Adaptive** 1/3	**Invariant + Alg. Repair**	**Model-Agnostic**	**See §4**

F5: Naive Majority-Voting Tyranny & Latency Blowup. Self-consistency pipelines [10, 4] sample $K \in [10, 20]$ completions and select the largest execution equivalence class. Whenever $> 50\%$ of samples omit `CAST(... AS REAL)` or `IS NOT NULL`, unweighted majority voting actively *overrides* the minority of dialect-correct queries while multiplying token cost and latency beyond interactive voice budgets.

3 The Argus-SQL Architecture

Argus-SQL addresses F1–F5 by pairing lightweight deterministic pre- and post-processing with targeted, complexity-gated LLM synthesis.

3.1 Stage 1: Voice & Intent Canonicalization ($\mathcal{C}_{\text{int}}$)

Given raw text or ASR transcript u and evidence e, $\mathcal{C}_{\text{int}}$ deterministically strips filler interjections and conversational wake-phrases, collapses consecutive word stutters ($w_i w_i \to w_i$), normalizes spoken cardinals adjacent to ranking/comparison operators (*"top five"* $\to$ `top 5`), expands high-frequency geographic shorthand into candidate entity spans $E(u)$, estimates expected projection arity $\hat{a} \geq 1$, and assigns a structural complexity tier $\tau(u, e) \in \{\textsc{Simple}, \textsc{Moderate}, \textsc{Challenging}\}$ based on ratio cues, superlative ordering, and multi-entity constraints.

3.2 Stage 2: 3-Gram Value Index & FK-Discounted Steiner Linking ($\mathcal{L}_{\text{link}}$)

Offline Character 3-Gram Inverted Index. To ground entity mentions without issuing per-query `LIKE` scans across large tables, we index distinct non-null `TEXT` literals once per database (bounded by $M_{\max} = 500$ distinct cells per column via a read-only URI connection). Each normalized cell value v is mapped to boundary-padded character trigrams $G_3(v) = \{v'[i : i + 3]\}$ in an inverted posting list. For each query span $s \in E(u)$, candidate cells sharing at least one trigram are scored by blending Jaccard overlap with Gestalt sequence similarity:

$$\text{Sim}(s, v) = \begin{cases} 1.0, & \tilde{s} = \tilde{v}, \\ \max\left(0.86, \rho_{\text{seq}}(\tilde{s}, \tilde{v})\right), & \text{pref}(\tilde{s}, \tilde{v}), \\ 0.45 J_3(\tilde{s}, \tilde{v}) + 0.55 \rho_{\text{seq}}(\tilde{s}, \tilde{v}), & \text{else}, \end{cases} \tag{2}$$

where $J_3(\tilde{s}, \tilde{v}) = |G_3(\tilde{s}) \cap G_3(\tilde{v})| / |G_3(\tilde{s}) \cup G_3(\tilde{v})|$. Top-$k$ matches exceeding $\theta_{\text{val}} = 0.45$ are injected directly into DDL column comments (`-- matched_values: ['California']`), adopting the compact code-representation insight of `DAIL-SQL` [3] and `CHESS` [8].

FK-Discounted Seed Selection & Steiner/Junction Closure. For each table $T \in \mathcal{T}$ and column $c \in T$, we compute a composite relevance score:

$$S(T, c) = S_{\text{lex}}(u, T, c) + 1.35\, S_{\text{ev}}(e, c) + 1.65\, S_{\text{val}}(T, c), \tag{3}$$

where S_{lex} applies morphological and compound subword tokenization (*e.g.*, splitting `gasstations` into `gas` + `station`) over column names and merged CSV value codebooks. Crucially, to prevent star-schema seed explosion (F3), if c is a primary key or surrogate foreign key (`*_id`) on a table T whose own name has zero lexical overlap with u and e, we apply a $0.25\times$ seed discount to S_{lex}. Furthermore, when an isolated orphan table ($d_{\text{FK}}(T) = 0$, such as a legacy backup table) matches only a subset of columns already covered by an FK-connected seed table, the orphan distractor is pruned. Finally, augmenting declared foreign keys with exact single-column entity-PK matches on schemas that omit DDL constraints, we compute all-pairs shortest paths over the undirected foreign-key graph $G_{\text{FK}} = (\mathcal{T}, \mathcal{E}_{\text{FK}})$ via Breadth-First Search and include many-to-many association tables referencing ≥ 2 selected tables plus compact lookup parents, retaining all columns on compact/medium tables (≤ 32 columns) and rendering composite keys as table-level `PRIMARY KEY` (c_1, c_2) constraints alongside representative SQLite cell examples.

3.3 Stage 3: Complexity-Gated Batch Synthesis ($\mathcal{G}_{\text{syn}}$)

Uniformly sampling $K = 20$ completions per question wastes latency and tokens on straightforward single-table lookups. **Argus-SQL** routes queries dynamically:

- **Voice Fast-Path ($K = 1$)**: When $\tau(u, e) = \textsc{Simple}$ and $e = \emptyset$, a single deterministic `formula_defensive` prompt is dispatched, preserving sub-second voice responsiveness.
- **Diverse Multi-Strategy Batch ($K = 3$)**: For Moderate and Challenging queries, three structurally distinct prompts—`decomposed_cot` (stepwise join and composite-key grain planning), `formula_defensive` (explicit SQLite cast, `NULL`-sort, and `DISTINCT` join rules), and `skeleton_minimal` (projection-minimal direct evidence translation)—are dispatched *concurrently* in a single deduplicated batch request.

3.4 Stage 4: Semantic Invariant Verification, Algebraic Repair & Consensus ($\mathcal{V}_{\text{arb}}$)

Hardened Read-Only SQLite Sandbox. Every unique SQL candidate is executed once inside a security-hardened SQLite sandbox opened via RFC 8089 `file:///<db>?mode=ro`, locked by a C-level `sqlite3_set_authorizer` allow-listing exclusively `SQLITE_SELECT`, `SQLITE_READ`, and safe scalar/aggregate functions (hard-denying `PRAGMA`, `ATTACH`, `DROP`, `DELETE`, `load_extension`, and blob memory bombs `zeroblob`/`randomblob`), and bounded by a virtual-machine opcode budget (5×10^5 VM steps) via `set_progress_handler` so runaway cross-joins abort in milliseconds.

Seven Semantic & Dialect Invariants. For each candidate q_i and execution outcome o_i, our verifier checks: (I1) `EXECUTION_ERROR`, (I2) `EMPTY_RESULT_ANOMALY` ($|o_i| = 0$ despite grounded cell matches), (I3) `ALL_NULL_RESULT_ANOMALY` ($o_i = [(\texttt{NULL}, \dots)]$), (I4) `INTEGER_DIVISION_TRUNCATION` (division `/` under ratio intent without `CAST(... AS REAL)`, `AVG`/`TOTAL`, or float-literal promotion), (I5) `UNFILTERED_NULL_ASC_LIMIT` (`ORDER BY col [ASC] LIMIT k` without `col IS NOT NULL`), (I6) `PROJECTION_ARITY_DRIFT` ($|\text{cols}(o_i) - \hat{a}| \geq 2$), and (I7) `JOIN_FANOUT_DUPLICATES` (multi-table `JOIN` entity listings returning $\geq 50\%$ duplicate rows without `DISTINCT`).

Zero-Token Algebraic Dialect Repair. Before invoking an expensive LLM repair pass, **Argus-SQL** applies literal-preserving algebraic rewrites to any candidate triggering I2, I4, I5, or I7: promoting uncast column, aggregate, or parenthesized arithmetic divisions `lhs / rhs` across `SELECT` and `ORDER BY` clauses to `CAST(lhs AS REAL) / NULLIF(rhs, 0)` (while preserving slashes inside quoted literals such as `'Free/Reduced'`), injecting outer-scope `col IS NOT NULL` guards prior to ascending `LIMIT` sorts, deduplicating 1-to-many join fanouts via `SELECT DISTINCT`, and substituting high-confidence grounded cell literals (Sim ≥ 0.75). Only if zero invariant-clean candidates exist after algebraic repair does **Argus-SQL** issue a single diagnostic LLM repair call seeded from the best executing candidate.

Invariant-Weighted Execution Consensus. Let $\mathcal{C}_r = \{i : \text{sig}(o_i) = r\}$ denote the equivalence cluster of candidates sharing normalized set/ordered result SHA-256 digest r. Each candidate receives weight $w_i = \gamma_{\text{err}}^{n_{\text{err}}(i)} \gamma_{\text{warn}}^{n_{\text{warn}}(i)}$ ($\gamma_{\text{err}} = 0.28, \gamma_{\text{warn}} = 0.65$, or 0 if execution failed), and each cluster is scored as:

$$\text{Score}(\mathcal{C}_r) = \sum_{i \in \mathcal{C}_r} w_i + \beta_{\text{div}} \big(|\text{Strategies}(\mathcal{C}_r)| - 1\big), \quad (4)$$

with $\beta_{\text{div}} = 0.15$. Under Eq. (4), even if 3/5 candidates share an integer-truncated result ($3 \times 0.28 = 0.84$), two invariant-clean candidates agreeing across distinct strategies ($2.0 + 0.15 = 2.15$) decisively win.

4 Experimental Evaluation

4.1 Experimental Setup & Metrics

We evaluate all systems using two official BIRD-SQL metrics [6] plus paired exact significance testing: (1) **Execution Accuracy (EX %)**, measuring relational set equivalence (or row-order-preserving equivalence when `ORDER BY` is present) between predicted and gold result sets with IEEE-754 floating-point tolerance; (2) **Reward-based Valid Efficiency Score (R-VES)**, applying the official BIRD Mini-Dev piecewise speed-ratio reward ($\{0, 0.25, 0.50, 0.75, 1.00, 1.25\}$ scaled to 0–100) over median interleaved warm sandbox runs; and (3) two-sided **exact paired McNemar binomial p-values** on discordant predictions.

Controlled Diagnostic Stress-Test (`Mini-BIRD` & `Voice-BIRD`). To isolate architectural mechanisms deterministically without confounding LLM API stochasticity, we construct three clean-room multi-table SQLite databases (`schools_mini`, `finance_mini`, and `clinical_mini`), each containing four tables: a parent entity table, a lexically opaque intermediate FK bridge table, a fact table with nullable scores and integer ratio columns, and a legacy orphan distractor table. Across these databases, we evaluate 24 stratified clean queries (8 Simple, 8 Moderate, 8 Challenging) and 24 paired **Voice-BIRD** spoken utter-

Table 2: Controlled diagnostic evaluation on Mini-BIRD (24 clean relational queries) and Voice-BIRD (24 spoken ASR-perturbed utterances) across progressive NL2SQL system architectures.

System / Configuration	Simple EX	Mod. EX	Chall. EX	Clean EX (%)	Clean R-VES	Voice EX (%)	Mean Tokens
B0_ZeroShotDDL	75.0	37.5	0.0	37.5	37.5	16.7	338
B1_DINDecomposed	100.0	25.0	0.0	41.7	41.7	25.0	316
B2_MACRefiner	100.0	37.5	0.0	45.8	45.8	25.0	318
B3_CHESSValueConsistency	100.0	62.5	0.0	54.2	54.2	41.7	1018
Argus-SQL	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	921

Table 3: Leave-one-out component ablation study of Argus-SQL across Clean and Voice-BIRD diagnostic splits, isolating each module's impact on Execution Accuracy (EX), R-VES, and prompt token budget.

System / Configuration	Simple EX	Mod. EX	Chall. EX	Clean EX (%)	Clean R-VES	Voice EX (%)	Mean Tokens
Argus-SQL	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	921
w/o Voice Canonicalizer	100.0	100.0	100.0	100.0	100.0	70.8	1018
w/o 3-Gram Value Grounding	100.0	75.0	50.0	75.0	75.0	62.5	886
w/o Steiner FK Closure	100.0	62.5	50.0	70.8	70.8	70.8	993
w/o Semantic Verifier & Repair	100.0	62.5	0.0	54.2	54.2	54.2	921
w/o Adaptive Fast-Path (Fixed K=3)	100.0	100.0	100.0	100.0	100.0	100.0	1018

ances generated by injecting ASR conversational preambles, mid-clause fillers, spoken number words, and geographic shorthand (*e.g.*, `'Calif'`, `'NY'`). All five systems (`B0_ZeroShotDDL`, `B1_DINDecomposed`, `B2_MACRefiner`, `B3_CHESSValueConsistency`, and **Argus-SQL**) execute over the exact same rule-based compositional DDL synthesizer (`SchemaGuidedDiagnosticSynthesizer`), ensuring that every accuracy difference emerges strictly from prompt DDL linking, voice canonicalization, candidate routing, and post-execution verification.

4.2 Comparative Progression & Ablation Analysis

Table 2 presents the comparative progression from `B0` to **Argus-SQL**:

1. **Schema Overload vs. Lexical Pruning (`B0` → `B1`)**: `B0_ZeroShotDDL` achieves 37.5% Clean EX (75.0% Simple EX) because unpruned legacy tables (`frpm_legacy`, `loans_legacy`) hijack table-agnostic aggregations. Lexical pruning (`B1`) eliminates orphan distractors on simple queries (100.0% Simple EX) but drops intermediate bridge tables on multi-hop moderate queries (25.0% Mod. EX).

2. **FK Closure & Syntax Refinement (`B1` → `B2`)**: `B2_MACRefiner` recovers clean multi-hop FK joins (45.8% Clean EX), yet fails whenever queries contain abbreviated cell literals (`'Calif'`) or silent SQLite arithmetic/`NULL` bugs that raise no SQLite error.

3. **3-Gram Value Grounding & Majority Trap (`B2` → `B3`)**: Adding character 3-gram cell retrieval (`B3`) lifts Moderate EX from 37.5% to 62.5% (54.2% overall Clean EX). However, on Challenging division queries, 2/3 non-defensive strategies emit raw integer division; naive majority voting selects the truncated `0` cluster (0.0% Challenging EX), while spoken `Voice-BIRD` utterances degrade `B3` by −12.5 points (41.7% Voice EX).

Table 4: Reproducible evaluation matrix for full public `BIRD-Dev` ($N = 1{,}534$) and `Mini-Dev` ($N = 500$) splits via `argus-ablation --mode bird`. Slots are left unpopulated in this clean-room repository artifact until executed against a live public LLM endpoint so zero empirical figures are ever extrapolated.

System	BIRD-Dev EX	Voice-BIRD EX	R-VES
B0_ZeroShotDDL	*[Run CLI]*	*[Run CLI]*	*[Run CLI]*
B1_DINDecomposed	*[Run CLI]*	*[Run CLI]*	*[Run CLI]*
B2_MACRefiner	*[Run CLI]*	*[Run CLI]*	*[Run CLI]*
B3_CHESSValueCons.	*[Run CLI]*	*[Run CLI]*	*[Run CLI]*
Argus-SQL	***[Run CLI]***	***[Run CLI]***	***[Run CLI]***

4. **The Argus-SQL Sweet Spot**: Combining voice canonicalization, FK-discounted Steiner/junction linking, 3-gram grounding, and semantic invariant repair/consensus achieves 100.0% EX and 100.0 R-VES across both Clean and Voice diagnostic splits ($p = 9.77 \times 10^{-4}$ Clean, $p = 1.22 \times 10^{-4}$ Voice, and $p = 5.96 \times 10^{-8}$ combined vs. `B3`) at a strictly lower token budget than `B3` (921 vs. 1,018 Clean, −9.5%; 920 vs. 1,044 Voice, −11.9%).

Table 3 confirms via leave-one-out ablations that every stage is load-bearing: disabling Semantic Verification & Algebraic Repair causes the sharpest drop (−45.8 EX points), followed by removing Steiner FK Closure (−29.2 points), 3-Gram Value Grounding (−25.0 Clean / −37.5 Voice points), and Voice Canonicalization (−29.2 Voice points), while disabling the Adaptive Fast-Path inflates mean token consumption from 921 to 1,018 (+10.5%) with zero accuracy benefit.

4.3 Full Public BIRD-Dev Evaluation Protocol

To uphold strict scientific integrity, we separate our deterministic diagnostic verification (Tables 2–3) from full-scale stochastic LLM evaluation on the 1,534-query public `BIRD-Dev` benchmark (Table 4). Our codebase natively ingests official `dev.json`, `dev_tables.json`, and `database_description/*.csv` files via `load_bird_split` and executes `B0`–`B3` and **Argus-SQL** over any public OpenAI-compatible or local open-weights endpoint (*e.g.*, `Qwen2.5-Coder-32B-Instruct`, `Llama-3.3-70B`, or `GPT-4o`) using batched concurrent HTTPS transport (`OpenAICompatibleBatchClient`) with SHA-256 prompt deduplication, automatically emitting `paper/tables/bird_live.tex`.

5 Discussion, Limitations & Future Work

Scaling from SQLite to Terabyte Warehouses. While BIRD-SQL standardizes on SQLite, deploying **Argus-SQL** over multi-terabyte cloud analytical engines introduces partition-pruning and scan-cost constraints. Our read-only execution sandbox and R-VES evaluator naturally generalize to `EXPLAIN` dry-run cost estimation: before executing $K = 3$ candidate queries over billion-row fact tables, candidates missing partition-column predicates can be penalized as an eighth semantic invariant (`UNPARTITIONED_FULL_SCAN`).

Limitations. First, our deterministic intent canonicalizer handles lexical ASR disfluencies and common abbreviations but cannot resolve phonetic homophones of rare proper nouns without acoustic N-best hypotheses from the speech recognizer. Second, when a user's spoken query is genuinely underspecified (*e.g.*, *"show me top performers"* on a schema containing five distinct performance metrics), silent SQL selection is inferior to asking a brief clarifying voice question—motivating entropy-triggered interactive disambiguation over our equivalence clusters $\mathcal{C}_r$.

6 Conclusion

By systematically dissecting how existing NL2SQL paradigms fail across clean and spoken relational queries, we demonstrated that high accuracy need not require brute-force 20-sample ensembles. **Argus-SQL** establishes a principled accuracy–latency sweet spot—uniting voice canonicalization, FK-discounted Steiner schema linking, character 3-gram value grounding, adaptive batch synthesis, and zero-token algebraic dialect verification—offering a clean, extensible foundation for the next generation of voice-driven data interfaces.

References

[1] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.

[2] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Lu Chen, Jinshu Lin, and Dongfang Lou. C3: Zero-shot text-to-SQL with ChatGPT. *arXiv preprint arXiv:2307.07306*, 2023.

[3] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5):1132–1145, 2024.

[4] Dongjun Lee, Choongwon Park, Jaehyuk Kim, and Heesoo Park. MCS-SQL: Leveraging multiple prompts and multiple-choice selection for text-to-SQL generation. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025.

[5] Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. Dawn of natural language to SQL: Are we fully ready? *Proceedings of the VLDB Endowment*, 17(11):3318–3331, 2024.

[6] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Binyuan Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 42330–42357, 2023.

[7] Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 36339–36348, 2023.

[8] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. CHESS: Contextual harnessing for efficient SQL synthesis. *arXiv preprint arXiv:2405.16755*, 2024.

[9] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, and Zhoujun Li. MAC-SQL: A multi-agent collaborative framework for text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025.

[10] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[11] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.

[12] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921, 2018.